

House Price EDA and Prediction

In this notebook we are going to predict the price of the houses which depends on many factors. This dataset is very useful for those people who have just finished learning basic regression techniques on a limited number of features and want to dive more into regression.

This notebook has been made after going through a lot of notebooks submitted by some pretty experienced and amazing people. I learnt a lot myself while i was making this notebook.

Let's start exploring the dataset and please upvote if you like it.

```
# This Python 3 environment comes with many helpful analytics  
libraries installed  
# It is defined by the kaggle/python Docker image:  
https://github.com/kaggle/docker-python  
# For example, here's several helpful packages to load  
  
import numpy as np # linear algebra  
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)  
  
# Input data files are available in the read-only "../input/"  
directory  
# For example, running this (by clicking run or pressing Shift+Enter)  
will list all files under the input directory  
  
import os  
for dirname, _, filenames in os.walk('/kaggle/input'):  
    for filename in filenames:  
        print(os.path.join(dirname, filename))  
  
import numpy as np # linear algebra  
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)  
import matplotlib.pyplot as plt  
import matplotlib.patches as mpatches  
import matplotlib.gridspec as gridspec  
from scipy import stats  
import seaborn as sns  
sns.set_style('whitegrid')  
  
import warnings  
warnings.simplefilter(action='ignore')  
  
from sklearn.metrics import mean_squared_error  
from sklearn.model_selection import cross_val_score, GridSearchCV,  
KFold, RandomizedSearchCV, train_test_split  
import math  
import sklearn.model_selection as ms
```

```

import sklearn.metrics as sklm

# You can write up to 20GB to the current directory (/kaggle/working/)
# that gets preserved as output when you create a version using "Save &
# Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
# be saved outside of the current session

/kaggle/input/house-prices-advanced-regression-techniques/
sample_submission.csv
/kaggle/input/house-prices-advanced-regression-techniques/data_descrip
tion.txt
/kaggle/input/house-prices-advanced-regression-techniques/train.csv
/kaggle/input/house-prices-advanced-regression-techniques/test.csv

train = pd.read_csv('/kaggle/input/house-prices-advanced-regression-
techniques/train.csv')
test = pd.read_csv('/kaggle/input/house-prices-advanced-regression-
techniques/test.csv')

test_id = test['Id']

df = train

df.head(5)

```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape
0	1	60	RL	65.0	8450	Pave	NaN	Reg
1	2	20	RL	80.0	9600	Pave	NaN	Reg
2	3	60	RL	68.0	11250	Pave	NaN	IR1
3	4	70	RL	60.0	9550	Pave	NaN	IR1
4	5	60	RL	84.0	14260	Pave	NaN	IR1

	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal
MoSold								
0	Lvl	AllPub	...	0	NaN	NaN	NaN	0
2								
1	Lvl	AllPub	...	0	NaN	NaN	NaN	0
5								
2	Lvl	AllPub	...	0	NaN	NaN	NaN	0
9								
3	Lvl	AllPub	...	0	NaN	NaN	NaN	0
2								
4	Lvl	AllPub	...	0	NaN	NaN	NaN	0
12								

	YrSold	SaleType	SaleCondition	SalePrice
0	2008	WD	Normal	208500
1	2007	WD	Normal	181500
2	2008	WD	Normal	223500
3	2006	WD	Abnorml	140000
4	2008	WD	Normal	250000

[5 rows x 81 columns]

```
df.drop('Id', axis = 1, inplace = True)
test.drop('Id', axis = 1, inplace = True)
```

```
train = df
```

```
print(f"Number of rows and number of columns in the train dataset are  
{train.shape[0]} and {train.shape[1]}")
print(f"Number of rows and number of columns in the test dataset are  
{test.shape[0]} and {test.shape[1]}")
```

Number of rows and number of columns in the train dataset are 1460 and 80

Number of rows and number of columns in the test dataset are 1459 and 79

There is 1 column less in the test dataset because we need to predict the price of the houses for the test dataset

SalePrice Feature Analysis

Let's Explore the target feature i.e SalesPrice.

```
def three_chart_plot(df, feature):
    fig = plt.figure(constrained_layout = True, figsize = (12, 8))
    grid = gridspec.GridSpec(ncols = 3, nrows = 3, figure = fig)

    ax1 = fig.add_subplot(grid[0, :2])
    ax1.set_title('Histogram')

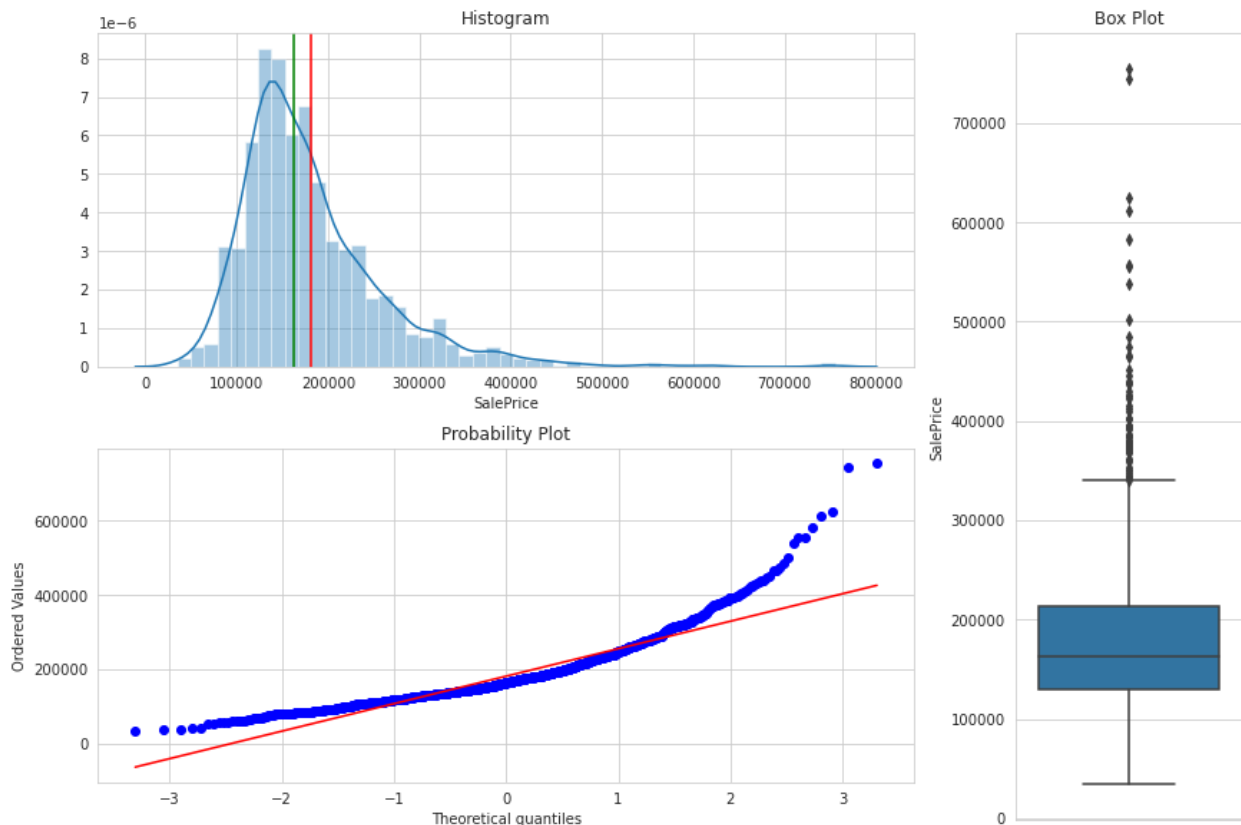
    sns.distplot(df.loc[:, feature], norm_hist = True, ax = ax1)
    plt.axvline(x = df[feature].mean(), c = 'red')
    plt.axvline(x = df[feature].median(), c = 'green')

    ax2 = fig.add_subplot(grid[1, :2])
    ax2.set_title('QQ_plot')
    stats.probplot(df.loc[:, feature], plot = ax2)

    ## Customizing the Box Plot.
```

```
ax3 = fig.add_subplot(grid[:, 2])
## Set title.
ax3.set_title('Box Plot')
sns.boxplot(df.loc[:,feature], orient = 'v', ax = ax3 )

three_chart_plot(df, 'SalePrice')
```



Red line in histogram indicates the mean of the SalePrice and the Green line indicates the median

From the above plots, we can observe that the Sales Price is not normally distributed. From the boxplot we can observe that the dataset have a number of outliers.

For the probability plot, the red line represents those points which would have been plotted for y-axis points **if those were normally distributed**. But the blue points represents the actual scenario. We can see that there is a lot of deviation on the both the ends i.e on the top right and bottom left.

```
from scipy.stats import skew
print("Skewness of the SalesPrice is", df['SalePrice'].skew())

Skewness of the SalesPrice is 1.8828757597682129
```

Skewness refers to the amount of asymmetry in the given feature or in other words amount of distortions from the normal distribution.

Here we can observe that the value of skewness is quite high which means that there is a large amount of asymmetry.

The peak of the histogram represents the mode i.e the price for which maximum number of houses were sold.

What kind of skewness is present in the given case?

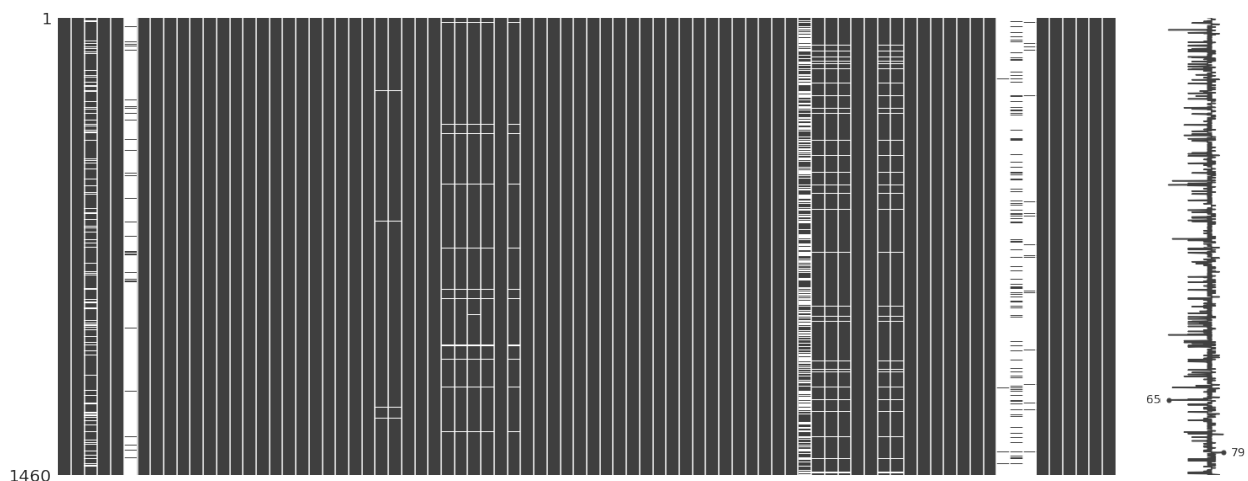
As the mean of the feature is greater than the median which is greater than the mode and the line is flat towards the right in the histogram, the given feature is **Positively Skewed**. Most of the houses were sold less than the average price.

NULL Values

There is an amazing library called missingno which helps us to visualize the number of Null values present in each feature.

```
import missingno as msno
%matplotlib inline
msno.matrix(train)

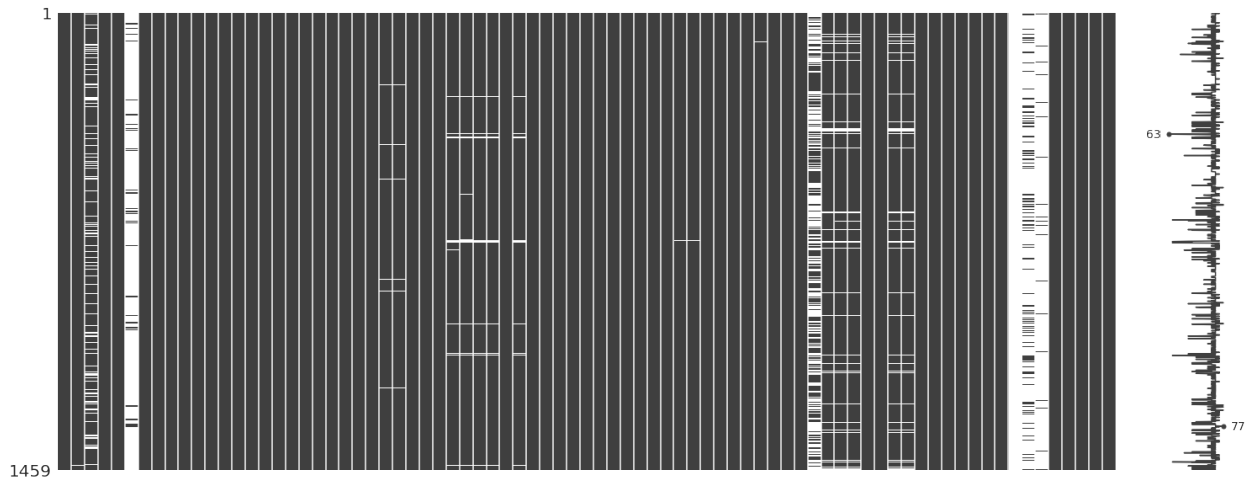
<matplotlib.axes._subplots.AxesSubplot at 0x7f3fcf412110>
```



Just by going through the plot we can see that there are a lot of NULL values

```
msno.matrix(test)

<matplotlib.axes._subplots.AxesSubplot at 0x7f3fcf3fd1d0>
```



```
def null_table(data):
    """
    A function which returns the number and percentage of null values
    in the given dataset.
    """
    indices = data.isnull().sum().index
    values = data.isnull().sum().values
    percentages = []
    for i in indices:
        percentages.append((data[i].isnull().sum() / data[i].shape[0])
* 100)
    d = {'Columns' : indices, 'Count of Null Values' : values,
'Approximate Percentage of Null Values' : percentages}
    # data = dict(zip(indices, percentages))
    null_frame = pd.DataFrame(data = d)
    return null_frame

null_frame_train = null_table(df)
null_frame_train.sort_values(by = 'Approximate Percentage of Null
Values').tail(10)
```

	Columns	Count of Null Values	Approximate Percentage of Null Values
62	GarageQual	81	5.547945
59	GarageFinish	81	5.547945
63	GarageCond	81	5.547945
57	GarageType	81	5.547945
2	LotFrontage	259	17.739726
56	FireplaceQu	690	47.260274

72	Fence	1179
80.753425		
5	Alley	1369
93.767123		
73	MiscFeature	1406
96.301370		
71	PoolQC	1453
99.520548		

```

null_frame_test = null_table(test)
null_frame_test.sort_values(by = 'Approximate Percentage of Null
Values').tail(10)

```

	Columns	Count of Null Values	Approximate Percentage of Null Values
62	GarageQual	78	
5.346127			
58	GarageYrBlt	78	
5.346127			
59	GarageFinish	78	
5.346127			
63	GarageCond	78	
5.346127			
2	LotFrontage	227	
15.558602			
56	FireplaceQu	730	
50.034270			
72	Fence	1169	
80.123372			
5	Alley	1352	
92.666210			
73	MiscFeature	1408	
96.504455			
71	PoolQC	1456	
99.794380			

We must observe here that **NA has a different meaning**. Here we can not impute or remove those rows having NA. For example if one of the values for Alley is NA, it means that home has no Alley or if the value of GarageType is NA it means that particular home has no Garage. That's why we need to take into account these values as it can affect the final prices of the house.

Let's combine both the train and test dataset and analyze the null values

```

na = train.shape[0]
nb = test.shape[0]
y_train = train['SalePrice'].to_frame()
# Combine train and test sets
total = pd.concat((train, test), sort = False).reset_index(drop =
True)
# Drop the target "SalePrice" and Id columns

```

```

total.drop(['SalePrice'], axis = 1, inplace = True)
print(f"Total size is {total.shape}")

Total size is (2919, 79)

def msv_1(data, thresh = 20, color = 'black', edgecolor = 'black',
height = 3, width = 15):

    plt.figure(figsize = (width, height))
    percentage = (data.isnull().mean()) * 100
    percentage.sort_values(ascending = False).plot.bar(color = color,
edgecolor = edgecolor)
    plt.axhline(y = thresh, color = 'r', linestyle = '-')

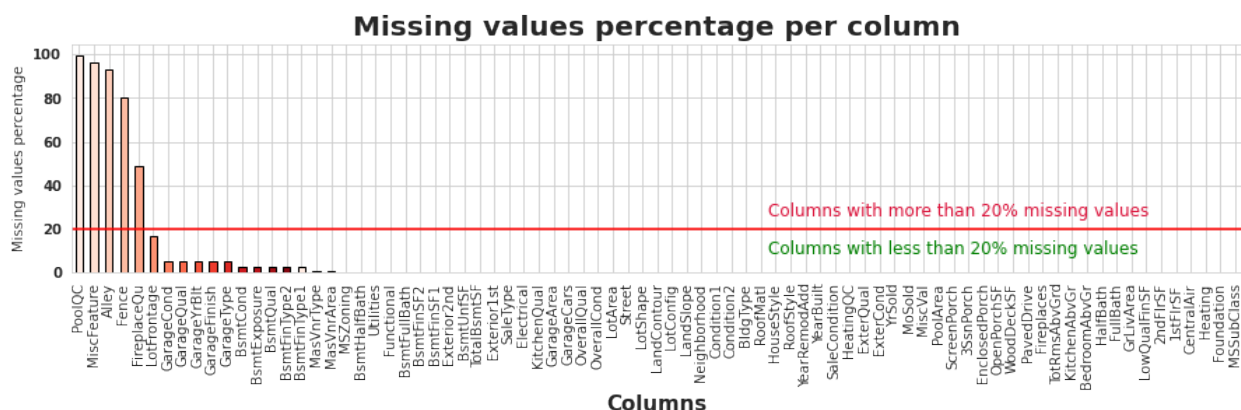
    plt.title('Missing values percentage per column', fontsize=20,
weight='bold' )

    plt.text(len(data.isnull().sum())/len(data))/1.7, thresh+12.5,
f'Columns with more than {thresh}% missing values', fontsize=12,
color='crimson',
            ha='left' ,va='top')
    plt.text(len(data.isnull().sum())/len(data))/1.7, thresh - 5,
f'Columns with less than {thresh}% missing values', fontsize=12,
color='green',
            ha='left' ,va='top')
    plt.xlabel('Columns', size=15, weight='bold')
    plt.ylabel('Missing values percentage')
    plt.yticks(weight = 'bold')

    return plt.show()

msv_1(total, 20, color=sns.color_palette('Reds',15))

```



There are 5 columns having more than 20 percent null values. We will deal with these values later.

Numerical and Categorical Variables

Exploring Numerical Variables

```
category_var = df.select_dtypes(include = 'object')
num_var = df.select_dtypes(exclude = 'object')

print("Number of categorical features are: ", category_var.shape[1])
print("Number of numerical features are: ", num_var.shape[1])
```

Number of categorical features are: 43
Number of numerical features are: 37

Before moving forward let's see the correlation of the numerical features with the target variable

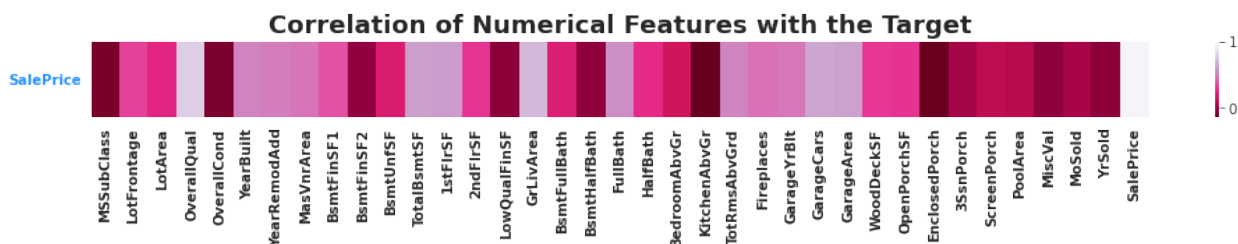
```
num = df.select_dtypes(exclude = 'object')
corr_num = num.corr()

f, ax = plt.subplots(figsize = (17, 1))

sns.heatmap(corr_num.sort_values(by = ['SalePrice'], ascending =
False).head(1), cmap='PuRd_r')

plt.title('Correlation of Numerical Features with the Target', weight
= 'bold', fontsize = 18)
plt.xticks(weight='bold')
plt.yticks(weight='bold', color='dodgerblue', rotation=0)

plt.show()
```



```
correlation = df.corr()['SalePrice'].sort_values(ascending =
False).to_frame().head(10)

cmap = sns.light_palette("cyan", as_cmap = True)

s = correlation.style.background_gradient(cmap = cmap)

s

<pandas.io.formats.style.Styler at 0x7f3fce8ad950>
```

From the above table, we can conclude that better the overall quality of the house, more will be the price of the house which is also true in real life. Same is true for the living area i.e as the area increases, more will be the Sale Price.

Let's plot a graph to examine the relationship between GrLivArea and the SalePrice

GrLivArea

```
plt.figure(figsize = ((15, 6)))
plt.scatter(x = 'GrLivArea', y = 'SalePrice', data = df,
color='crimson', alpha=0.4)
plt.title('Relationship between Living Area and Sale Price', weight =
'bold', fontsize = 10)
plt.xlabel('Area of the house', weight = 'bold')
plt.ylabel('Sale Price', weight = 'bold')
plt.show()
```



We can observe that there is almost **a linear relationship between Living Area and the Sale Price**. If the area is huge then price should also have also been high but the two rightmost points suggest something else. **These two points are outliers**. We will drop both of them below.

```
train = train[train['GrLivArea'] < 4500]
previous_train = df
df = train

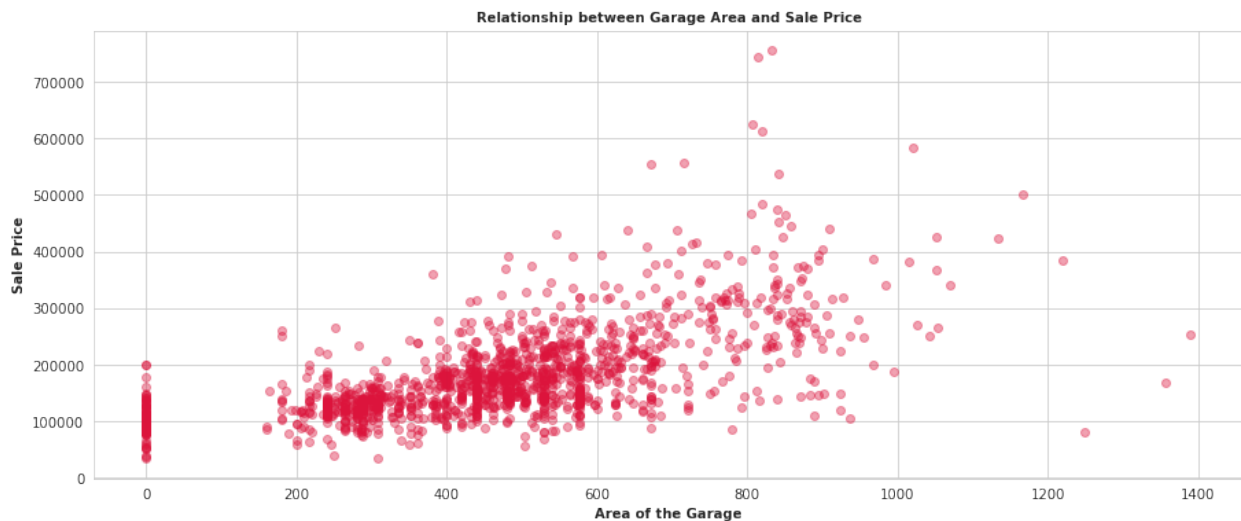
plt.figure(figsize = ((15, 6)))
plt.scatter(x = 'GrLivArea', y = 'SalePrice', data = df,
color='crimson', alpha=0.4)
plt.title('Relationship between Living Area and Sale Price', weight =
'bold', fontsize = 10)
plt.xlabel('Area of the house', weight = 'bold')
plt.ylabel('Sale Price', weight = 'bold')
plt.show()
```



We can observe that those two values have been successfully dropped.

Garage Area

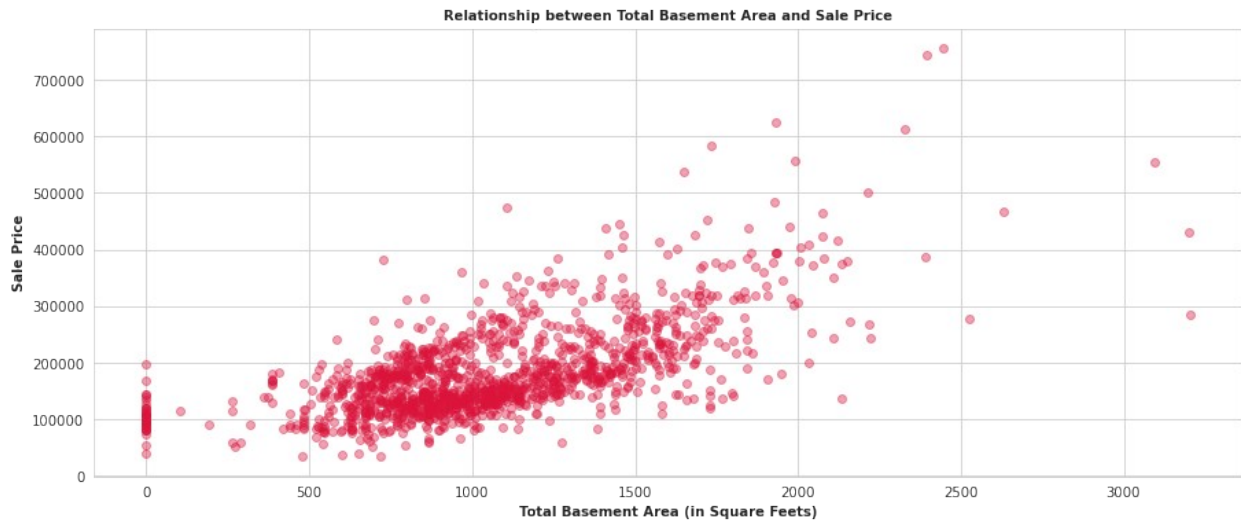
```
plt.figure(figsize = ((15, 6)))
plt.scatter(x = 'GarageArea', y = 'SalePrice', data = df,
color='crimson', alpha=0.4)
plt.title('Relationship between Garage Area and Sale Price', weight =
'bold', fontsize = 10)
plt.xlabel('Area of the Garage', weight = 'bold')
plt.ylabel('Sale Price', weight = 'bold')
plt.show()
```



Total Basement Area

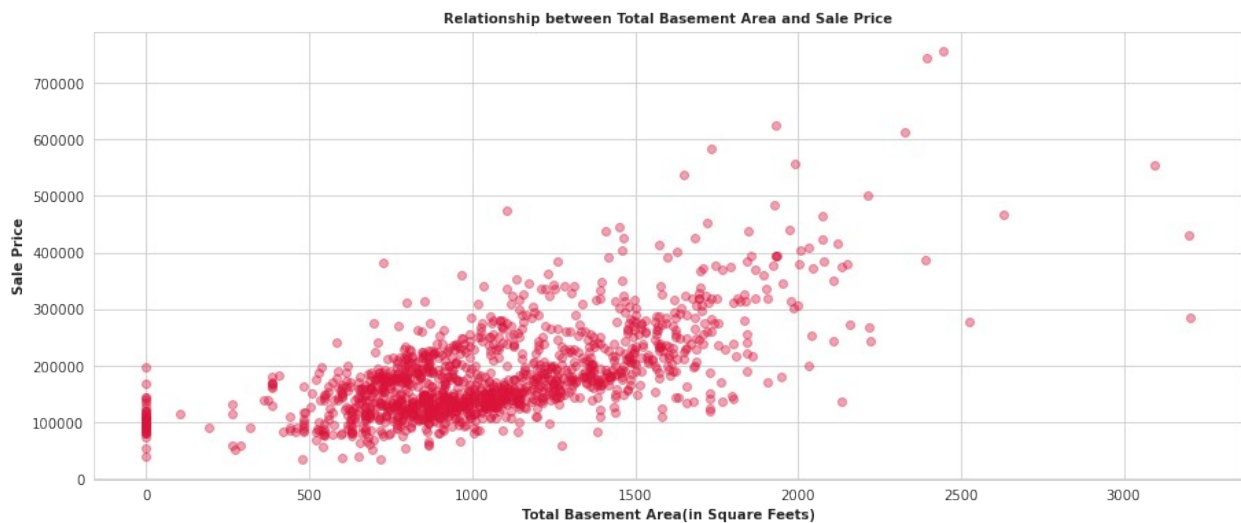
```
plt.figure(figsize = ((15, 6)))
plt.scatter(x = 'TotalBsmtSF', y = 'SalePrice', data = df, color =
'crimson', alpha = 0.4)
```

```
plt.title('Relationship between Total Basement Area and Sale Price',
weight = 'bold', fontsize = 10)
plt.xlabel('Total Basement Area (in Square Feets)', weight = 'bold')
plt.ylabel('Sale Price', weight = 'bold')
plt.show()
```



1st Floor Area

```
plt.figure(figsize = ((15, 6)))
plt.scatter(x = 'TotalBsmtSF', y = 'SalePrice', data = df, color =
'crimson', alpha = 0.4)
plt.title('Relationship between Total Basement Area and Sale Price',
weight = 'bold', fontsize = 10)
plt.xlabel('Total Basement Area(in Square Feets)', weight = 'bold')
plt.ylabel('Sale Price', weight = 'bold')
plt.show()
```



Before we move forward we need to understand the assumptions of linear regression:

- Linearity
- Homoscedasticity
- No or little Multicollinearity
- Independence of Error

Since we fit a linear model, we assume that the relationship is linear, and the errors, or residuals, are pure random fluctuations around the true line. We expect that the variability in the dependent variable doesn't increase as the value of the independent increases, which is the assumption of equal variance, also known as Homoscedasticity. We also assume that the observations are independent of one another (No Multicollinearity), and a correlation between sequential observations or auto-correlation is not there.

Now, these assumptions are prone to happen altogether. In other words, if we see one of these assumptions in the dataset, it's more likely that we may come across with others mentioned above. Therefore, we can find and fix various assumptions with a few unique techniques.

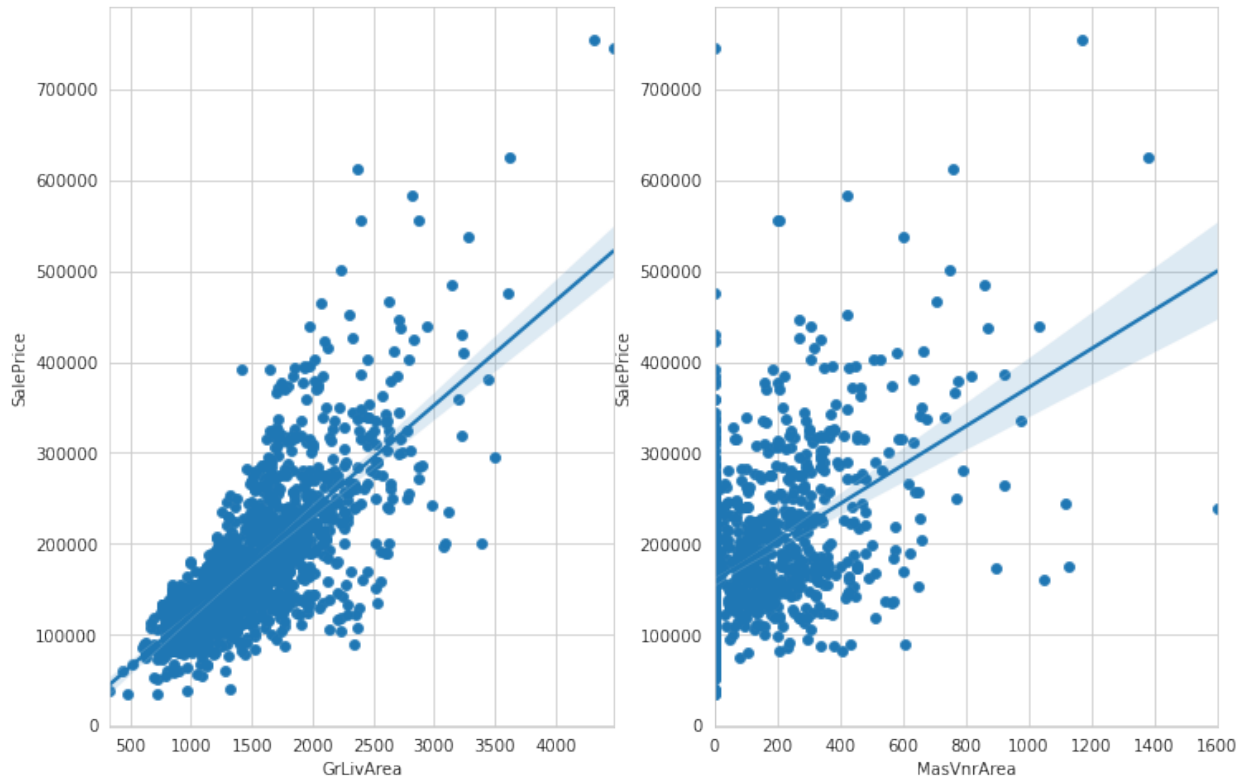
In order to discover the linearity let's plot scatter plots for GrLivArea and MasVnrArea

```
## Scatter plotting for SalePrice and GrLivArea.
fig, (ax1, ax2) = plt.subplots(figsize = (12, 8), ncols = 2, sharey =
False)

sns.scatterplot(x = df['GrLivArea'], y = df['SalePrice'], ax = ax1)
## Putting a regression line.
sns.regplot(x = df['GrLivArea'], y = df['SalePrice'], ax = ax1)

sns.scatterplot(x = df['MasVnrArea'], y = df['SalePrice'], ax = ax2)
## regression line for MasVnrArea and SalePrice.
sns.regplot(x = df['MasVnrArea'], y = df['SalePrice'], ax=ax2)

<matplotlib.axes._subplots.AxesSubplot at 0x7f3fcedbae10>
```

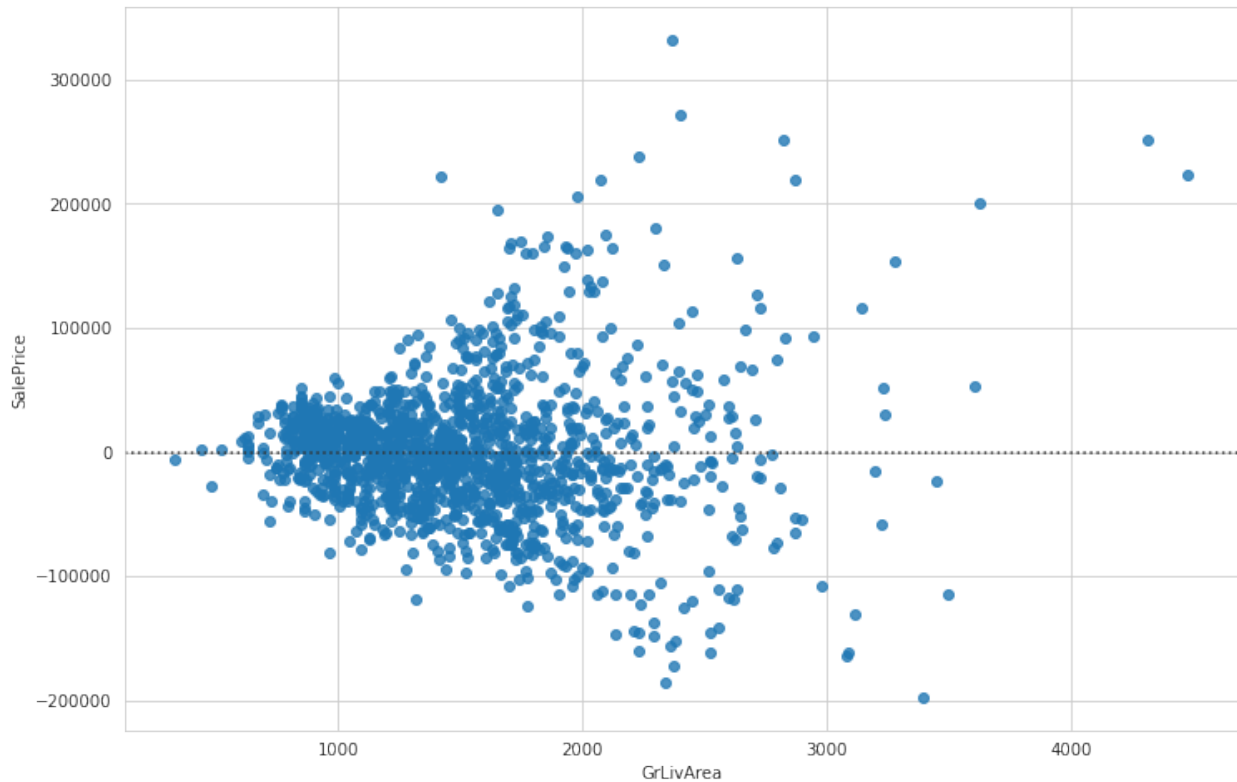


We can observe that the relationship between Sales Price and GrLivArea is much more linear than the relationship between Sales Price and MasVnrArea.

Let's look at the residual plot for independent variable GrLivArea and our target variable SalePrice.

A residual value is a measure of how much a regression line vertically misses a data point. Regression lines are the best fit of a set of data. You can think of the lines as averages; a few data points will fit the line and others will miss. A residual plot has the Residual Values on the vertical axis; the horizontal axis displays the independent variable.

```
plt.subplots(figsize = (12,8))
sns.residplot(df['GrLivArea'], df['SalePrice'])
<matplotlib.axes._subplots.AxesSubplot at 0x7f3fcac8c110>
```

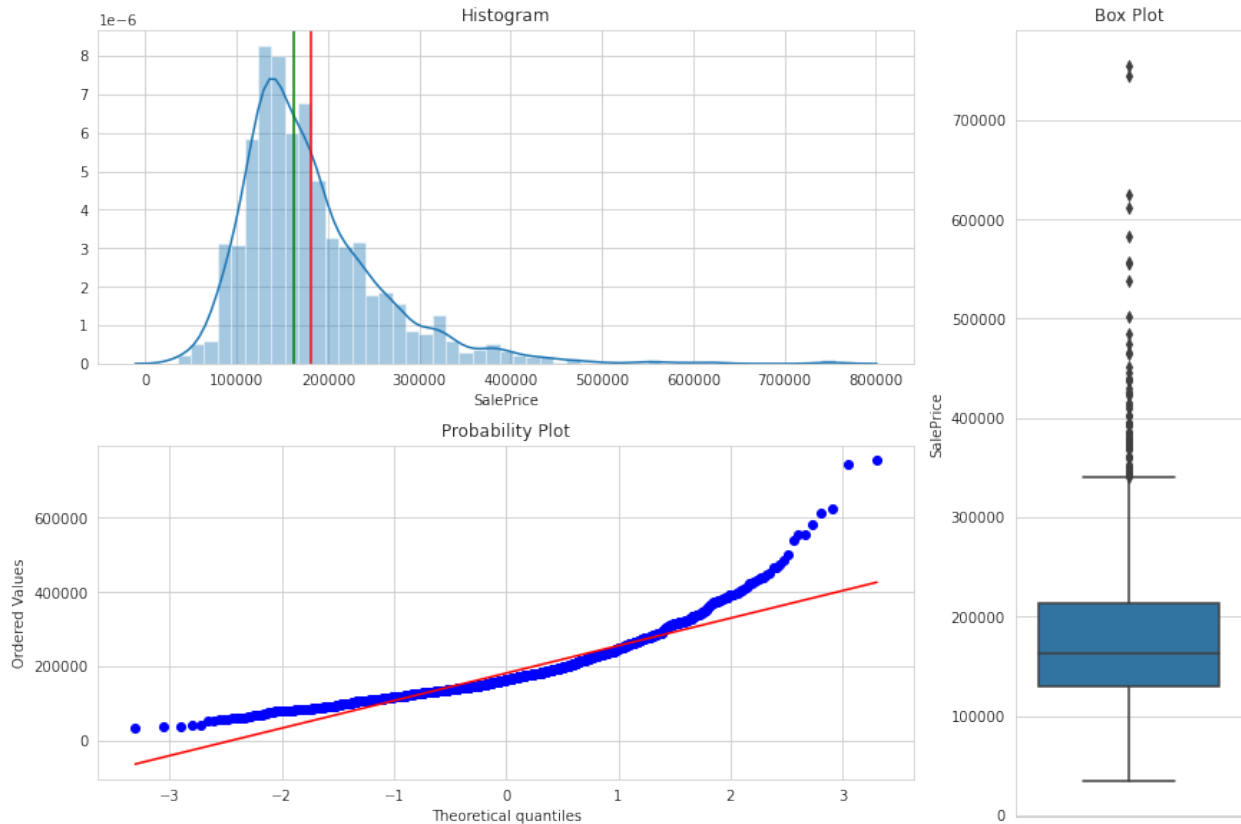


Ideally, if the assumptions are met, the residuals will be randomly scattered around the centerline of zero with no apparent pattern. The residual will look like an unstructured cloud of points centered around zero. However, our residual plot is anything but an unstructured cloud of points. Even though it seems like there is a linear relationship between the response variable and predictor variable, the residual plot looks more like a funnel.

The error plot shows that as GrLivArea value increases, the variance also increases, which is the characteristics known as Heteroscedasticity. The linear regression analysis requires the dependent variable to be multivariate normally distributed. A histogram, box plot, or a Q-Q-Plot can check if the target variable is normally distributed.

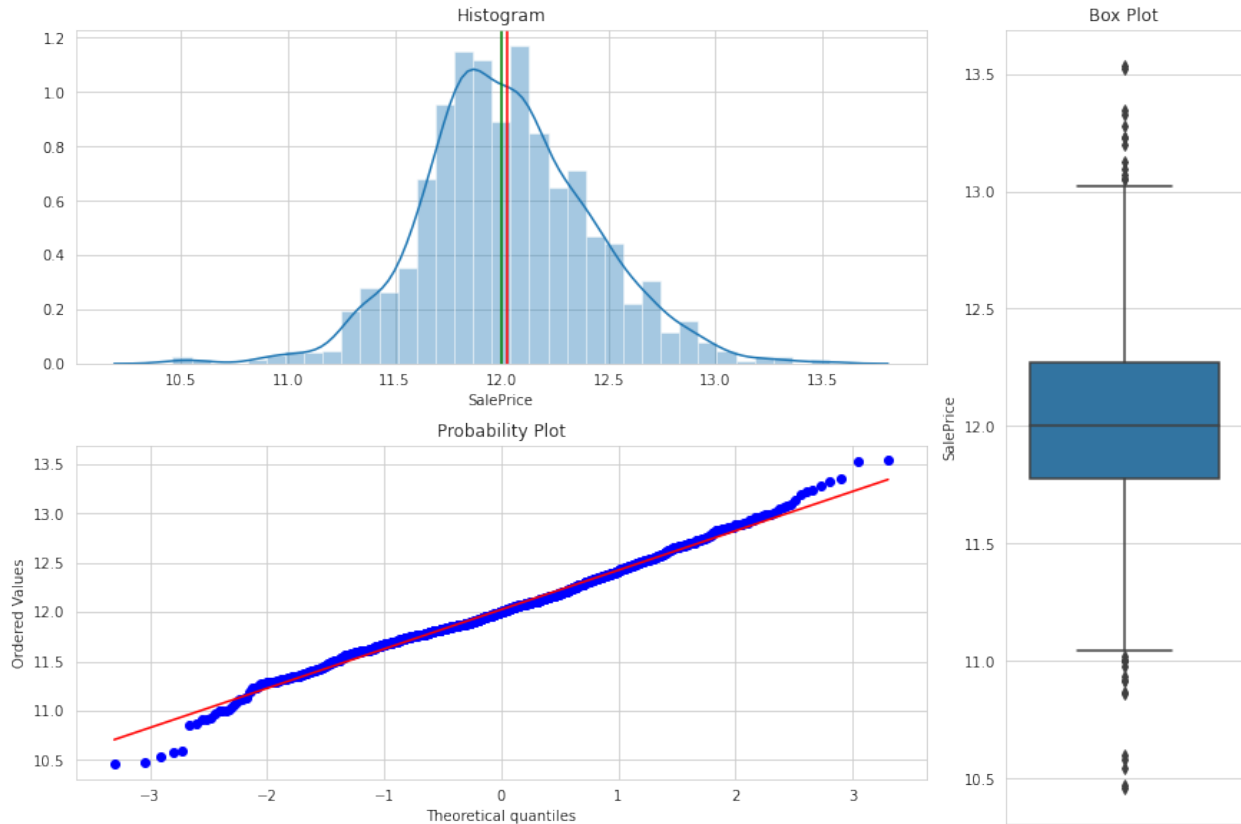
Let's plot the three graphs again for the target feature.

```
three_chart_plot(df, 'SalePrice')
```



Now let's make sure that the target variable follows a normal distribution. In order to make a feature follow normal distribution, we will perform log transformation.

```
train["SalePrice"] = np.log1p(train["SalePrice"])
df = train
three_chart_plot(df, 'SalePrice')
```

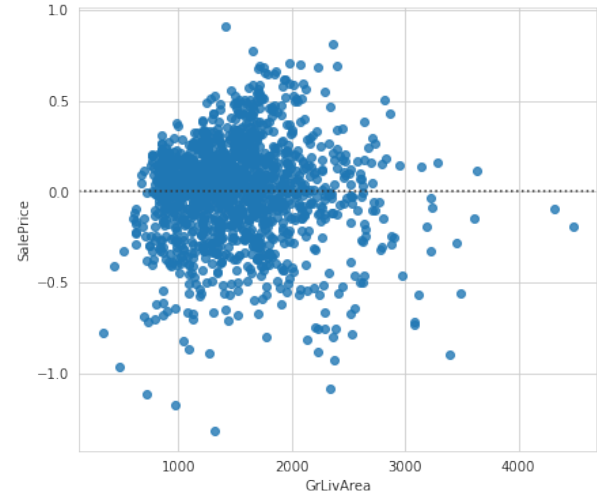
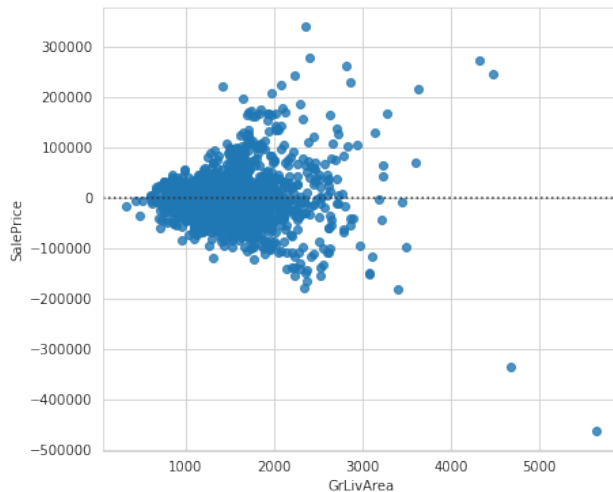



We can observe that the SalesPrice feature now follows a normal distribution as the Histogram resembles bell-shape and the QQ-plot also overlaps with the red line. Let's again plot the scatter plot for GrLivArea and SalePrice to see whether Heteroscedasticity was removed.

```
fig, (ax1, ax2) = plt.subplots(figsize = (15, 6),
                                ncols = 2,
                                sharey = False,
                                sharex = False)

sns.residplot(previous_train['GrLivArea'],
              previous_train['SalePrice'], ax = ax1)
sns.residplot(df['GrLivArea'], df['SalePrice'], ax = ax2)

<matplotlib.axes._subplots.AxesSubplot at 0x7f3fca032490>
```



We can see that the scatterplot on the left had heteroscedasticity (funnel like shape) but after applying log transformation to the feature, it was removed and now it's Homoscedastic.

Linear Regression indicates significant relationships between the dependent variable and the independent variable. It assumes that there is a linear relationship between the independent variables and the dependent variable but this is not always the case in real life. There are very less scenarios where this assumption holds true. Multiple Regression also suffers from multicollinearity, auto-correlation, and heteroskedasticity.

The interpretation of the linear coefficient is that it represents the mean change in the dependent variable for 1 unit change in the independent variable when all the other independent variables are held constant.

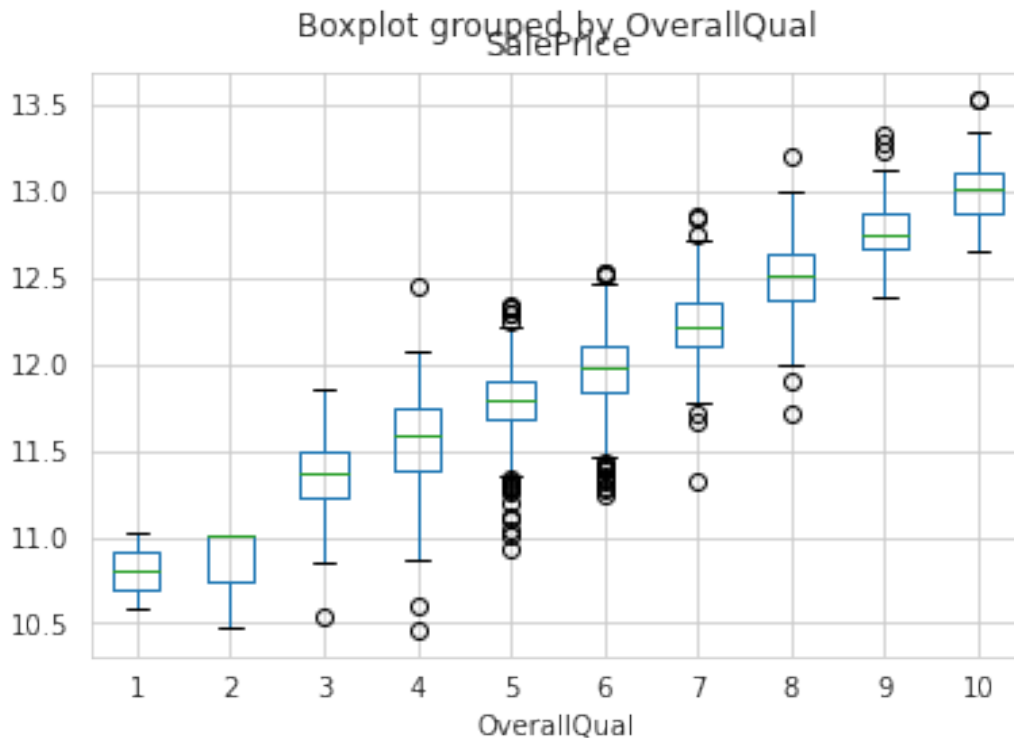
Now suppose that there are two correlated independent variables (A and B) and we are aiming to find the right coefficient for these independent variables. When the coefficient for A is calculated, will that coefficient be accurate? No, it won't be right because changes in A are associated with shifts in B, and as we have already discussed that all the other variables need to be held constant, B can't be held as a constant because A and B are correlated. This is known as Multicollinearity and it is one of the disadvantages of Linear Regression.

In order to solve this problem, we use various regularization techniques (l1, l2 etc) or other kind of regression techniques like Elastic Net Regression, Lasso and Ridge Regression which automatically takes care of multicollinearity.

Check this [blog](#) for better understanding

```
plt.figure(figsize = ((15, 10)))
df.boxplot(by = 'OverallQual', column = 'SalePrice')
plt.show()
```

<Figure size 1080x720 with 0 Axes>



As the quality increases, price of the houses also increase

Exploring Categorical Variables

```
def horizontal_plot(data, column_name, color, plot_title):
    """
    A function that plots a horizontal bar graph for the given column
    name
    """

    fig, ax = plt.subplots(figsize = (9, 6))

    title_cnt = data[column_name].value_counts().sort_values(ascending
= False).reset_index()

    mn = ax.barh(title_cnt.iloc[:, 0], title_cnt.iloc[:, 1], edgecolor
= 'black', color =
sns.color_palette(color, len(title_cnt)))

    for s in ['top', 'bottom', 'left', 'right']:
        ax.spines[s].set_visible(False)

    # Remove x,y Ticks
    ax.xaxis.set_ticks_position('none')
    ax.yaxis.set_ticks_position('none')
```

```

# Add padding between axes and labels
ax.xaxis.set_tick_params(pad=5)
ax.yaxis.set_tick_params(pad=10)

# Add x,y gridlines
ax.grid(b=True, color='grey', linestyle='-.', linewidth=1,
alpha=0.2)

# Show top values
ax.invert_yaxis()

ax.set_title(plot_title, weight='bold', loc='center', pad=10,
fontsize=16)

ax.set_xlabel('Count', weight='bold')

for i in ax.patches:
    ax.text(i.get_width() + 1.0, i.get_y() + 0.5,
str(round((i.get_width()), 2)),
        fontsize=10, fontweight='bold', color='grey')

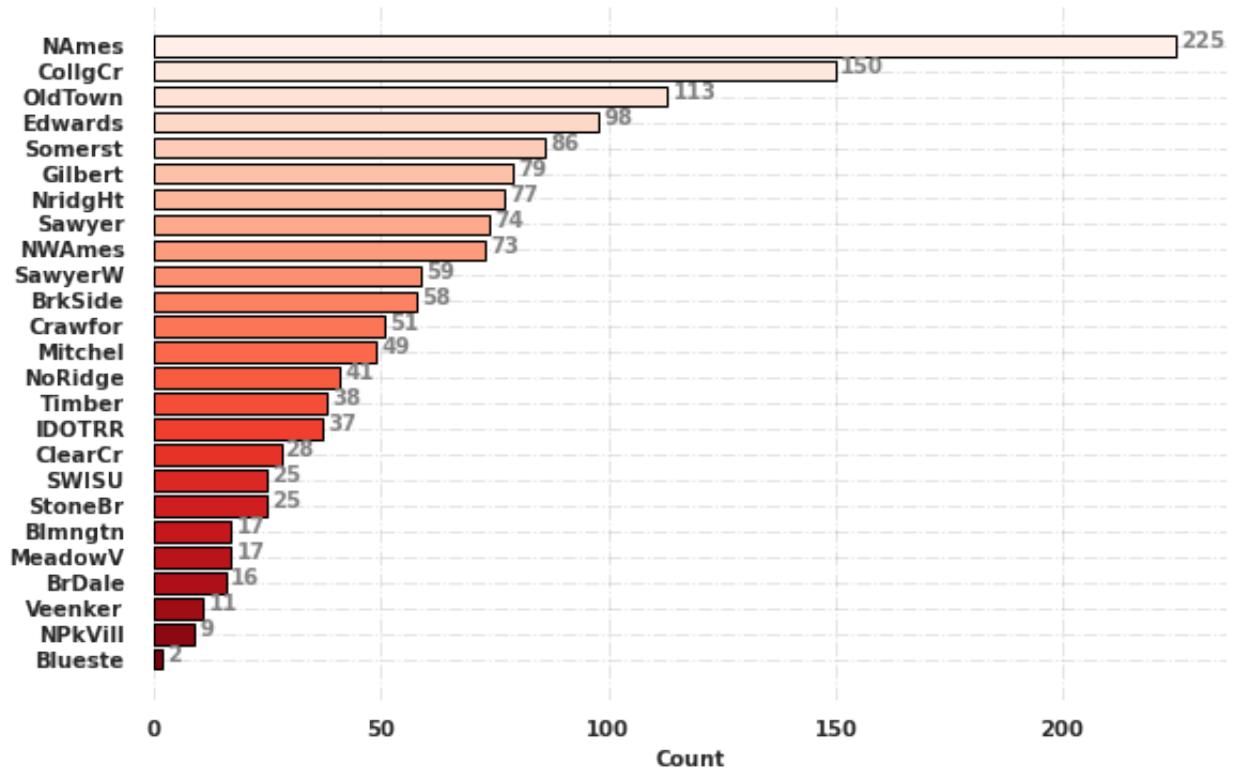
plt.yticks(weight='bold')
plt.xticks(weight='bold')

plt.show()

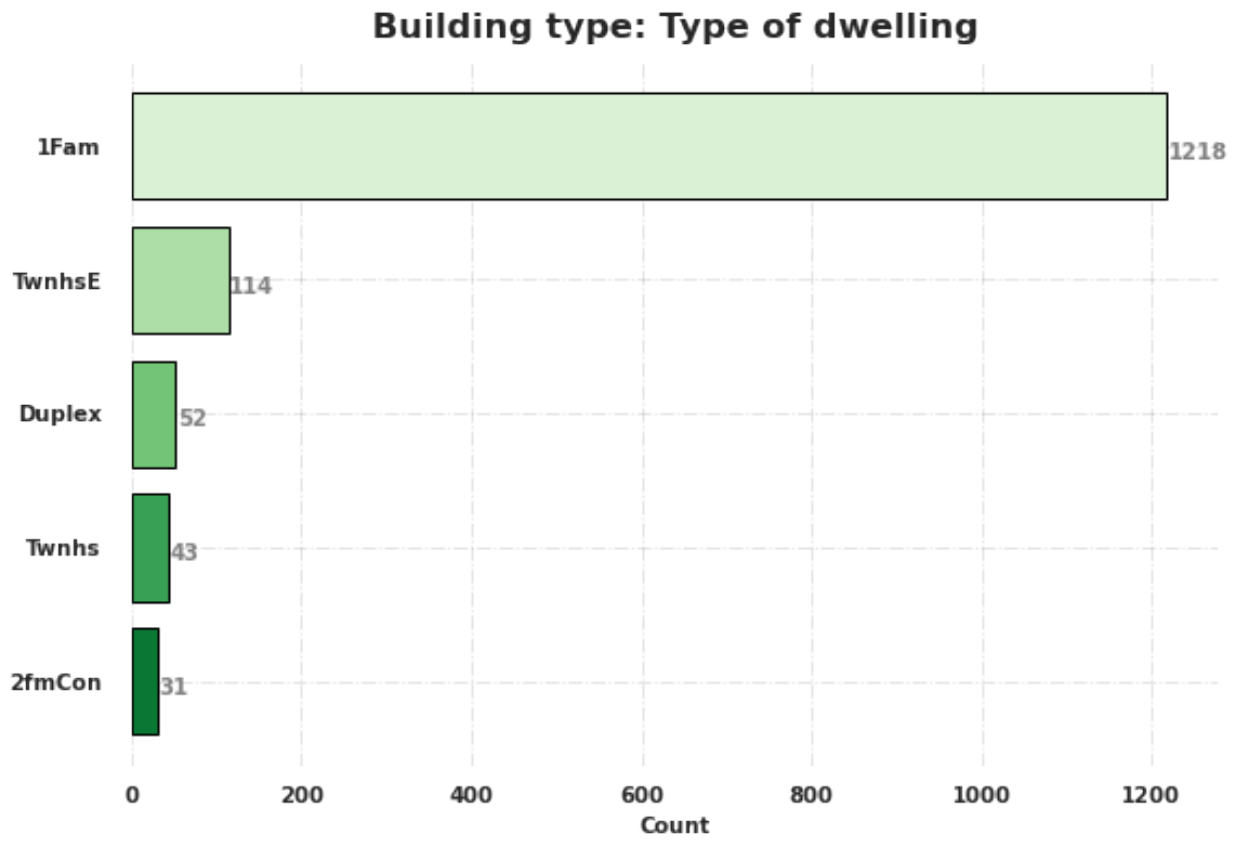
horizontal_plot(df, 'Neighborhood', 'Reds', 'Most frequent
neighborhoods')

```

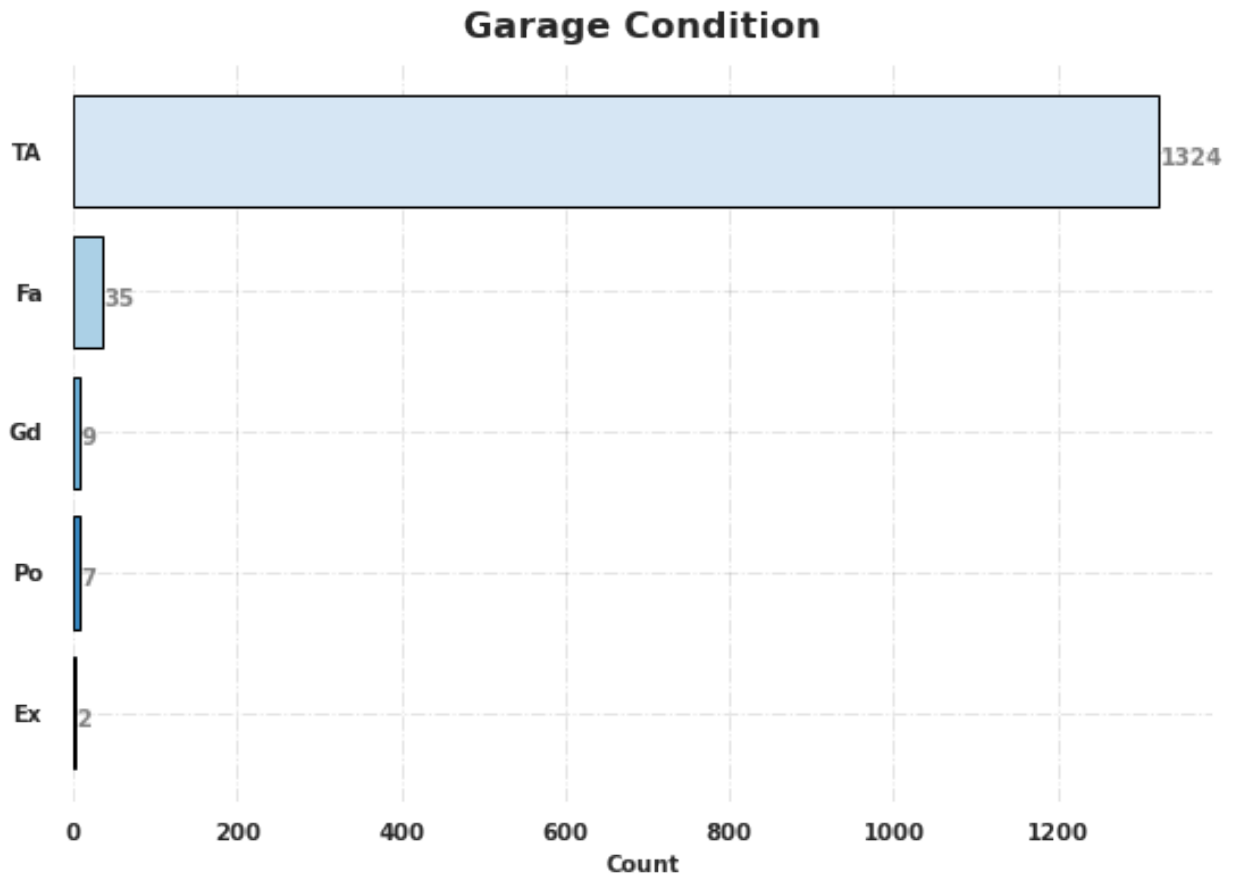
Most frequent neighborhoods



```
horizontal_plot(df, 'BldgType', 'Greens', 'Building type: Type of dwelling')
```

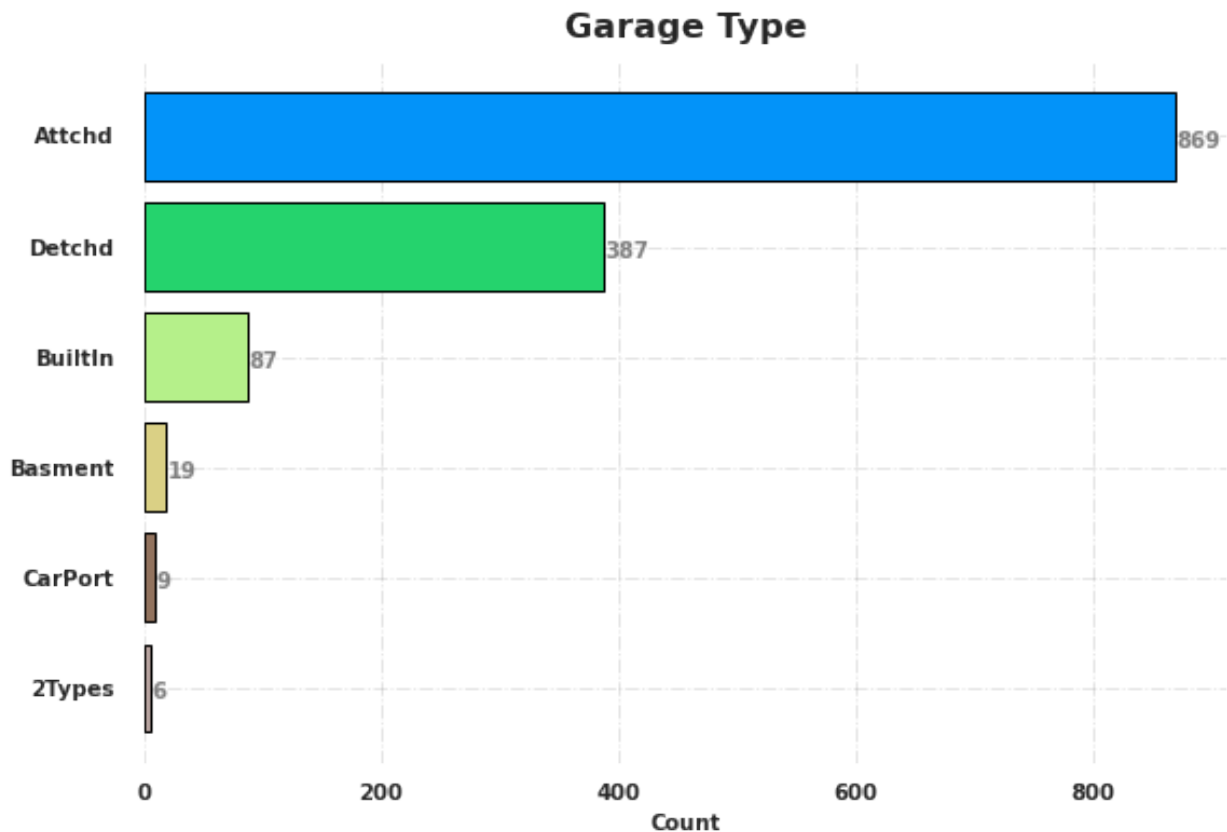


```
horizontal_plot(df, 'GarageCond', 'Blues', 'Garage Condition')
```



Most of the houses have Typically Average Garage Condition. Only couple of houses have Excellent Garage Condition.

```
horizontal_plot(df, 'GarageType', 'terrain', 'Garage Type')
```



Most of the houses have Attached Garages and only 19 have Garages in the Basement.

Feature Engineering

First let's combine the train and test features.

```
y = df['SalePrice'].reset_index(drop = True)
train_features = df.drop('SalePrice', axis = 1)
test_features = test.copy()

features = pd.concat((train_features, test_features)).reset_index(drop = True)

features.shape

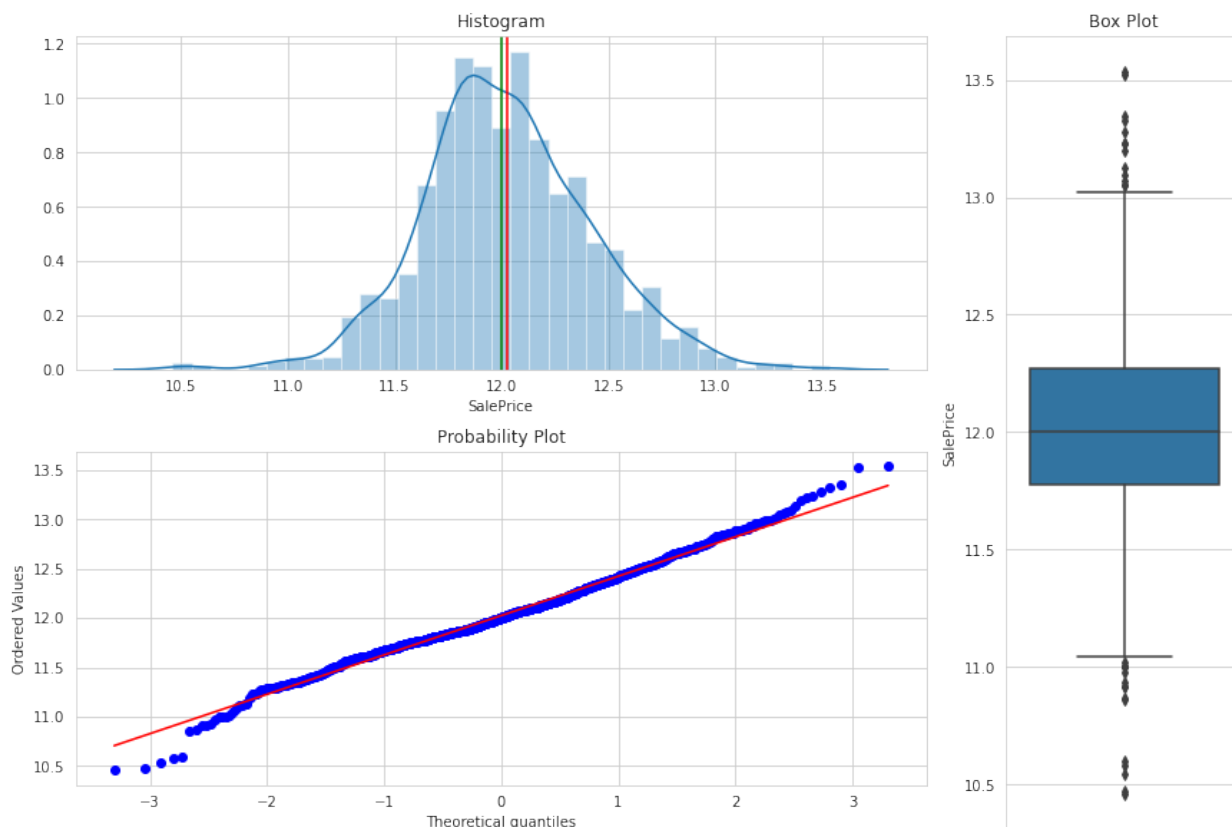
(2917, 79)
```

Some of the variables given in the description should be of string data type but instead they are stored as numerical data type. First let's change them into string data type


```
features['MSSubClass'] = features['MSSubClass'].apply(str)
features['MoSold'] = features['MoSold'].astype(str)
features['YrSold'] = features['YrSold'].astype(str)
```

We have already normalized the target variable in the above section. Let's plot it again

```
three_chart_plot(df, 'SalePrice')
```



Let's see the null values statistics of the combined dataset.

```
null_table(features).sort_values(by = 'Approximate Percentage of Null Values').tail(10)
```

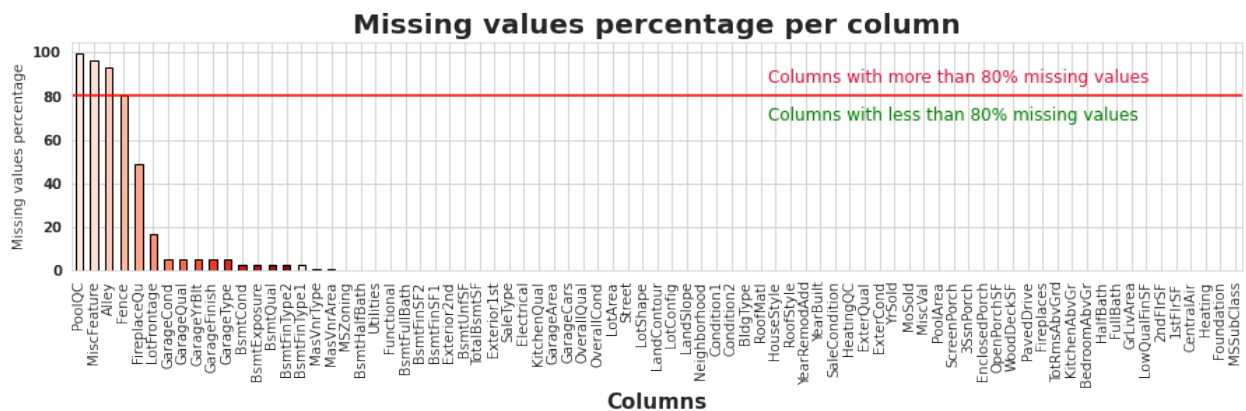
	Columns	Count of Null Values	Approximate Percentage of Null Values
59	GarageFinish	159	5.450806
58	GarageYrBlt	159	5.450806
62	GarageQual	159	5.450806
63	GarageCond	159	5.450806
2	LotFrontage	486	

```

16.660953
56 FireplaceQu 1420
48.680151
72 Fence 2346
80.425094
5 Alley 2719
93.212204
73 MiscFeature 2812
96.400411
71 PoolQC 2908
99.691464

```

```
msv_1(features, 80, color = sns.color_palette('Reds',15))
```



We can see that there are 4 features having more than 80% missing values. We will drop these 4 features.

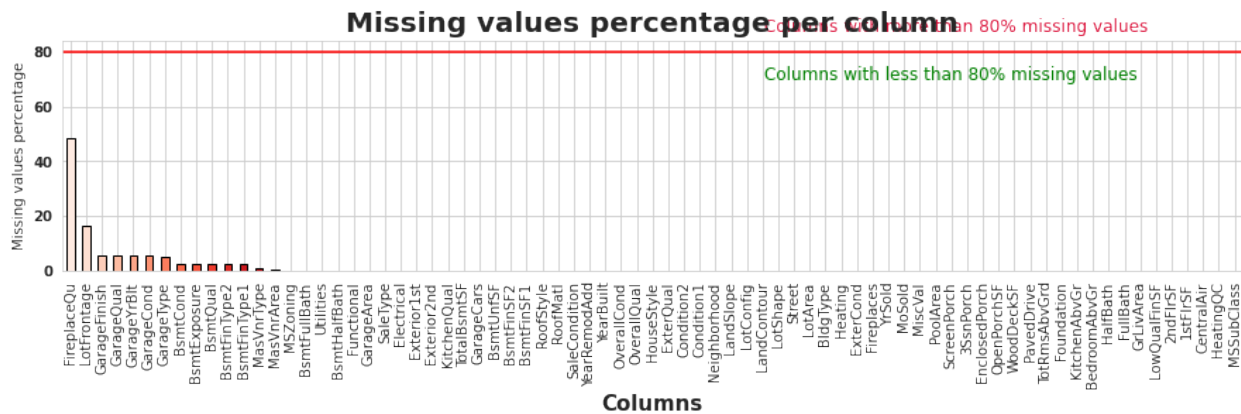
```

features = features.drop(['PoolQC', 'MiscFeature', 'Alley', 'Fence'],
axis = 1)
features.shape

(2917, 75)

msv_1(features, 80, color = sns.color_palette('Reds',15))

```



Now none of the features have above 80% missing values.

We must observe here that **NA has a different meaning**. Here we can not impute or remove those rows having NA. For example if one of the values for Alley is NA, it means that home has no Alley or if the value of GarageType is NA it means that particular home has no Garage. That's why we need to take into account these values as it can affect the final prices of the house.

I have identified some of the features which have NA as a category, they are listed in form of a list below. We will replace these null values with None.

Reading through the description of the features is very important, otherwise the above points can't be concluded.

```
description = open("/kaggle/input/house-prices-advanced-regression-techniques/data_description.txt", "r")
print(description.read())
```

MSSubClass: Identifies the type of dwelling involved in the sale.

```
20 1-STORY 1946 & NEWER ALL STYLES
30 1-STORY 1945 & OLDER
40 1-STORY W/FINISHED ATTIC ALL AGES
45 1-1/2 STORY - UNFINISHED ALL AGES
50 1-1/2 STORY FINISHED ALL AGES
60 2-STORY 1946 & NEWER
70 2-STORY 1945 & OLDER
75 2-1/2 STORY ALL AGES
80 SPLIT OR MULTI-LEVEL
85 SPLIT FOYER
90 DUPLEX - ALL STYLES AND AGES
120 1-STORY PUD (Planned Unit Development) - 1946 & NEWER
150 1-1/2 STORY PUD - ALL AGES
160 2-STORY PUD - 1946 & NEWER
180 PUD - MULTILEVEL - INCL SPLIT LEV/FOYER
190 2 FAMILY CONVERSION - ALL STYLES AND AGES
```

MSZoning: Identifies the general zoning classification of the sale.

A	Agriculture
C	Commercial
FV	Floating Village Residential
I	Industrial
RH	Residential High Density
RL	Residential Low Density
RP	Residential Low Density Park
RM	Residential Medium Density

LotFrontage: Linear feet of street connected to property

LotArea: Lot size in square feet

Street: Type of road access to property

Grvl	Gravel
Pave	Paved

Alley: Type of alley access to property

Grvl	Gravel
Pave	Paved
NA	No alley access

LotShape: General shape of property

Reg	Regular
IR1	Slightly irregular
IR2	Moderately Irregular
IR3	Irregular

LandContour: Flatness of the property

Lvl	Near Flat/Level
Bnk	Banked - Quick and significant rise from street grade to building
HLS	Hillside - Significant slope from side to side
Low	Depression

Utilities: Type of utilities available

AllPub	All public Utilities (E,G,W,& S)
NoSewr	Electricity, Gas, and Water (Septic Tank)
NoSeWa	Electricity and Gas Only
ELO	Electricity only

LotConfig: Lot configuration

Inside	Inside lot
Corner	Corner lot

CulDSac Cul-de-sac
FR2 Frontage on 2 sides of property
FR3 Frontage on 3 sides of property

LandSlope: Slope of property

Gtl Gentle slope
Mod Moderate Slope
Sev Severe Slope

Neighborhood: Physical locations within Ames city limits

Blmngtn	Bloomington Heights
Blueste	Bluestem
BrDale	Briardale
BrkSide	Brookside
ClearCr	Clear Creek
CollgCr	College Creek
Crawfor	Crawford
Edwards	Edwards
Gilbert	Gilbert
IDOTRR	Iowa DOT and Rail Road
MeadowV	Meadow Village
Mitchel	Mitchell
Names	North Ames
NoRidge	Northridge
NPkVill	Northpark Villa
NridgHt	Northridge Heights
NWAmes	Northwest Ames
OldTown	Old Town
SWISU	South & West of Iowa State University
Sawyer	Sawyer
SawyerW	Sawyer West
Somerst	Somerset
StoneBr	Stone Brook
Timber	Timberland
Veenker	Veenker

Condition1: Proximity to various conditions

Artery	Adjacent to arterial street
Feedr	Adjacent to feeder street
Norm	Normal
RRNn	Within 200' of North-South Railroad
RRAn	Adjacent to North-South Railroad
PosN	Near positive off-site feature--park, greenbelt, etc.
PosA	Adjacent to postive off-site feature
RRNe	Within 200' of East-West Railroad
RRAe	Adjacent to East-West Railroad

Condition2: Proximity to various conditions (if more than one is present)

Artery	Adjacent to arterial street
Feedr	Adjacent to feeder street
Norm	Normal
RRNn	Within 200' of North-South Railroad
RRAn	Adjacent to North-South Railroad
PosN	Near positive off-site feature--park, greenbelt, etc.
PosA	Adjacent to postive off-site feature
RRNe	Within 200' of East-West Railroad
RRAe	Adjacent to East-West Railroad

BldgType: Type of dwelling

1Fam	Single-family Detached
2FmCon	Two-family Conversion; originally built as one-family dwelling
Duplx	Duplex
TwnhsE	Townhouse End Unit
TwnhsI	Townhouse Inside Unit

HouseStyle: Style of dwelling

1Story	One story
1.5Fin	One and one-half story: 2nd level finished
1.5Unf	One and one-half story: 2nd level unfinished
2Story	Two story
2.5Fin	Two and one-half story: 2nd level finished
2.5Unf	Two and one-half story: 2nd level unfinished
SFoyer	Split Foyer
SLvl	Split Level

OverallQual: Rates the overall material and finish of the house

10	Very Excellent
9	Excellent
8	Very Good
7	Good
6	Above Average
5	Average
4	Below Average
3	Fair
2	Poor
1	Very Poor

OverallCond: Rates the overall condition of the house

10	Very Excellent
9	Excellent
8	Very Good
7	Good
6	Above Average
5	Average
4	Below Average
3	Fair
2	Poor
1	Very Poor

YearBuilt: Original construction date

YearRemodAdd: Remodel date (same as construction date if no remodeling or additions)

RoofStyle: Type of roof

Flat	Flat
Gable	Gable
Gambrel	Gabrel (Barn)
Hip Hip	
Mansard	Mansard
Shed	Shed

RoofMatl: Roof material

ClyTile	Clay or Tile
CompShg	Standard (Composite) Shingle
Membran	Membrane
Metal	Metal
Roll	Roll
Tar&Grv	Gravel & Tar
WdShake	Wood Shakes
WdShngl	Wood Shingles

Exterior1st: Exterior covering on house

AsbShng	Asbestos Shingles
AsphShn	Asphalt Shingles
BrkComm	Brick Common
BrkFace	Brick Face
CBlock	Cinder Block
CemntBd	Cement Board
HdBoard	Hard Board
ImStucc	Imitation Stucco
MetalSd	Metal Siding
Other	Other
Plywood	Plywood
PreCast	PreCast

Stone	Stone
Stucco	Stucco
VinylSd	Vinyl Siding
Wd Sdng	Wood Siding
WdShing	Wood Shingles

Exterior2nd: Exterior covering on house (if more than one material)

AsbShng	Asbestos Shingles
AsphShn	Asphalt Shingles
BrkComm	Brick Common
BrkFace	Brick Face
CBlock	Cinder Block
CemntBd	Cement Board
HdBoard	Hard Board
ImStucc	Imitation Stucco
MetalSd	Metal Siding
Other	Other
Plywood	Plywood
PreCast	PreCast
Stone	Stone
Stucco	Stucco
VinylSd	Vinyl Siding
Wd Sdng	Wood Siding
WdShing	Wood Shingles

MasVnrType: Masonry veneer type

BrkCmn	Brick Common
BrkFace	Brick Face
CBlock	Cinder Block
None	None
Stone	Stone

MasVnrArea: Masonry veneer area in square feet

ExterQual: Evaluates the quality of the material on the exterior

Ex	Excellent
Gd	Good
TA	Average/Typical
Fa	Fair
Po	Poor

ExterCond: Evaluates the present condition of the material on the exterior

Ex	Excellent
Gd	Good
TA	Average/Typical

Fa Fair
Po Poor

Foundation: Type of foundation

BrkTil Brick & Tile
CBlock Cinder Block
PConc Poured Contrete
Slab Slab
Stone Stone
Wood Wood

BsmtQual: Evaluates the height of the basement

Ex Excellent (100+ inches)
Gd Good (90-99 inches)
TA Typical (80-89 inches)
Fa Fair (70-79 inches)
Po Poor (<70 inches)
NA No Basement

BsmtCond: Evaluates the general condition of the basement

Ex Excellent
Gd Good
TA Typical - slight dampness allowed
Fa Fair - dampness or some cracking or settling
Po Poor - Severe cracking, settling, or wetness
NA No Basement

BsmtExposure: Refers to walkout or garden level walls

Gd Good Exposure
Av Average Exposure (split levels or foyers typically score average or above)
Mn Mimimum Exposure
No No Exposure
NA No Basement

BsmtFinType1: Rating of basement finished area

GLQ Good Living Quarters
ALQ Average Living Quarters
BLQ Below Average Living Quarters
Rec Average Rec Room
LwQ Low Quality
Unf Unfinished
NA No Basement

BsmtFinSF1: Type 1 finished square feet

BsmtFinType2: Rating of basement finished area (if multiple types)

GLQ	Good Living Quarters
ALQ	Average Living Quarters
BLQ	Below Average Living Quarters
Rec	Average Rec Room
LwQ	Low Quality
Unf	Unfinished
NA	No Basement

BsmtFinSF2: Type 2 finished square feet

BsmtUnfSF: Unfinished square feet of basement area

TotalBsmtSF: Total square feet of basement area

Heating: Type of heating

Floor	Floor Furnace
GasA	Gas forced warm air furnace
GasW	Gas hot water or steam heat
Grav	Gravity furnace
OthW	Hot water or steam heat other than gas
Wall	Wall furnace

HeatingQC: Heating quality and condition

Ex	Excellent
Gd	Good
TA	Average/Typical
Fa	Fair
Po	Poor

CentralAir: Central air conditioning

N	No
Y	Yes

Electrical: Electrical system

SBrkr	Standard Circuit Breakers & Romex
FuseA	Fuse Box over 60 AMP and all Romex wiring (Average)
FuseF	60 AMP Fuse Box and mostly Romex wiring (Fair)
FuseP	60 AMP Fuse Box and mostly knob & tube wiring (poor)
Mix	Mixed

1stFlrSF: First Floor square feet

2ndFlrSF: Second floor square feet

LowQualFinSF: Low quality finished square feet (all floors)

GrLivArea: Above grade (ground) living area square feet

BsmtFullBath: Basement full bathrooms

BsmtHalfBath: Basement half bathrooms

FullBath: Full bathrooms above grade

HalfBath: Half baths above grade

Bedroom: Bedrooms above grade (does NOT include basement bedrooms)

Kitchen: Kitchens above grade

KitchenQual: Kitchen quality

Ex Excellent

Gd Good

TA Typical/Average

Fa Fair

Po Poor

TotRmsAbvGrd: Total rooms above grade (does not include bathrooms)

Functional: Home functionality (Assume typical unless deductions are warranted)

Typ Typical Functionality

Min1 Minor Deductions 1

Min2 Minor Deductions 2

Mod Moderate Deductions

Maj1 Major Deductions 1

Maj2 Major Deductions 2

Sev Severely Damaged

Sal Salvage only

Fireplaces: Number of fireplaces

FireplaceQu: Fireplace quality

Ex Excellent - Exceptional Masonry Fireplace

Gd Good - Masonry Fireplace in main level

TA Average - Prefabricated Fireplace in main living area or
Masonry Fireplace in basement

Fa Fair - Prefabricated Fireplace in basement

Po Poor - Ben Franklin Stove

NA No Fireplace

GarageType: Garage location

2Types	More than one type of garage
Attchd	Attached to home
Basment	Basement Garage
BuiltIn	Built-In (Garage part of house - typically has room above garage)
CarPort	Car Port
Detchd	Detached from home
NA	No Garage

GarageYrBlt: Year garage was built

GarageFinish: Interior finish of the garage

Fin	Finished
Rfn	Rough Finished
Unf	Unfinished
NA	No Garage

GarageCars: Size of garage in car capacity

GarageArea: Size of garage in square feet

GarageQual: Garage quality

Ex	Excellent
Gd	Good
TA	Typical/Average
Fa	Fair
Po	Poor
NA	No Garage

GarageCond: Garage condition

Ex	Excellent
Gd	Good
TA	Typical/Average
Fa	Fair
Po	Poor
NA	No Garage

PavedDrive: Paved driveway

Y	Paved
P	Partial Pavement
N	Dirt/Gravel

WoodDeckSF: Wood deck area in square feet

OpenPorchSF: Open porch area in square feet

EnclosedPorch: Enclosed porch area in square feet

3SsnPorch: Three season porch area in square feet

ScreenPorch: Screen porch area in square feet

PoolArea: Pool area in square feet

PoolQC: Pool quality

Ex	Excellent
Gd	Good
TA	Average/Typical
Fa	Fair
NA	No Pool

Fence: Fence quality

GdPrv	Good Privacy
MnPrv	Minimum Privacy
GdWo	Good Wood
MnWw	Minimum Wood/Wire
NA	No Fence

MiscFeature: Miscellaneous feature not covered in other categories

Elev	Elevator
Gar2	2nd Garage (if not described in garage section)
Othr	Other
Shed	Shed (over 100 SF)
TenC	Tennis Court
NA	None

MiscVal: \$Value of miscellaneous feature

MoSold: Month Sold (MM)

YrSold: Year Sold (YYYY)

SaleType: Type of sale

WD	Warranty Deed - Conventional
CWD	Warranty Deed - Cash
VWD	Warranty Deed - VA Loan
New Home	just constructed and sold
COD	Court Officer Deed/Estate
Con	Contract 15% Down payment regular terms
ConLw	Contract Low Down payment and low interest

ConLI	Contract Low Interest
ConLD	Contract Low Down
0th	Other

SaleCondition: Condition of sale

Normal	Normal Sale
Abnorml	Abnormal Sale - trade, foreclosure, short sale
AdjLand	Adjoining Land Purchase
Alloca	Allocation - two linked properties with separate deeds, typically condo with a garage unit
Family	Sale between family members
Partial	Home was not completed when last assessed (associated with New Homes)

```
def filling_null(features):

    # In the description for 'Functional Feature', it is given that
Typ refers to the Typical Functionality
    # We will replace null values in the 'Functional' feature with
'Typ'

    features['Functional'] = features['Functional'].fillna('Typ')

    # Replace the null values in the below columns with their mode. We
are replacing with mode because
    # all of these features are categorical variable, we can't take
the mean nor the median.

    features['Electrical'] =
features['Electrical'].fillna(features['Electrical'].mode()[0])

    features['KitchenQual'] =
features['KitchenQual'].fillna(features['KitchenQual'].mode()[0])

    features['Exterior1st'] =
features['Exterior1st'].fillna(features['Exterior1st'].mode()[0])

    features['Exterior2nd'] =
features['Exterior2nd'].fillna(features['Exterior2nd'].mode()[0])

    features['SaleType'] =
features['SaleType'].fillna(features['SaleType'].mode()[0])

    # In order to fill the null values for MSZoning, we will take into
account the MSSubClass featue.
    # We are doing so because the type of dwelling in a given area
largely affects the zone of the area.
```

```

features['MSZoning'] = features.groupby('MSSubClass')
['MSZoning'].transform(lambda x: x.fillna(x.mode()[0]))

# In order to fill the null values for LotFrontage(Linear feet of
street connected to property)
# we will fill it with median of the values grouped by
Neighborhood.

features['LotFrontage'] = features.groupby('Neighborhood')
['LotFrontage'].transform(lambda x: x.fillna(x.median()))

# As mentioned before that the 'NA' value in many columns
represent the absence of that feature for
# a particular house, we will replace those null values with None.
# 'None' for categorical feature and 0 for numerical feature.

cat_features = features.select_dtypes(include = 'object').columns
num_features = features.select_dtypes(exclude = 'object').columns

features[cat_features] = features[cat_features].fillna('None')
features[num_features] = features[num_features].fillna(0)

return features

features = filling_null(features)
features.shape

(2917, 75)

features.isnull().sum().sum()

0

```

Now there are 0 null values in the given dataset

Now let's create some new features.

Feature Creation

```

def new_features(features):

    features['HasPool'] = features['PoolArea'].apply(lambda x: 1 if x
> 0 else 0)
    features['Has2ndFloor'] = features['2ndFlrSF'].apply(lambda x : 1
if x > 0 else 0)
    features['HasGarage'] = features['GarageArea'].apply(lambda x: 1
if x > 0 else 0)
    features['HasBsmt'] = features['TotalBsmtSF'].apply(lambda x: 1 if
x > 0 else 0)

```

```

    features['HasFireplace'] = features['Fireplaces'].apply(lambda x:
1 if x > 0 else 0)

    # Adding total sqfootage features
    # In order to get the total area of the house we have to add the
    basement area, 1st floor area and 2nd floor area.

    features['TotalSF'] = features['TotalBsmtSF'] +
features['1stFlrSF'] + features['2ndFlrSF']

    # Total number of bathrooms = number of full bathrooms + 0.5 *
    number of half bathrooms
    # + number of bathrooms in basement + 0.5 * number of half
    bathrooms in the basement.

    features['Total_Bathrooms'] = (features['FullBath'] + (0.5 *
features['HalfBath']) +
                                features['BsmtFullBath'] + (0.5 *
features['BsmtHalfBath']))

    features['Total_porch_sf'] = (features['OpenPorchSF'] +
features['3SsnPorch'] +
                                features['EnclosedPorch'] +
features['ScreenPorch'])

    # Add years since remodel
    features['YearsSinceRemodel'] = features['YrSold'].astype(int) -
features['YearRemodAdd'].astype(int)

    return features

```

We have added certain features which are quite intuitive and are taken into consideration in real life too while buying the house. Like what's the total area in square feet of the house or how many bathrooms are there in the house. We could also look for the availability of swimming pool, garage, fireplace and basement in the house. That's why I have added such features in the dataset.

```

features = new_features(features)
features.shape

(2917, 84)

```

Let's analyse the skewness of the numerical features.

```

num_features = features.select_dtypes(exclude = 'object').columns

skewed_features = features[num_features].apply(lambda x :
x.skew()).sort_values(ascending = False)
skew_features = pd.DataFrame({'Skew Value' : skewed_features})

```



```
skew_features.head(10)
```

	Skew Value
MiscVal	21.950962
PoolArea	17.697766
HasPool	15.502729
LotArea	13.116240
LowQualFinSF	12.090757
3SsnPorch	11.377932
KitchenAbvGr	4.302763
BsmtFinSF2	4.146636
EnclosedPorch	4.004404
ScreenPorch	3.947131

There's is quite a lot of asymmetry in the above features. We will need to fix the skewness

```
def fix_skew(features):  
  
    from scipy.special import boxcox1p  
    from scipy.stats import boxcox_normmax  
  
    ## Getting all the data that are not of "object" type.  
    numerical_columns = features.select_dtypes(exclude =  
'object').columns  
  
    # Check the skew of all numerical features  
    skewed_features = features[numerical_columns].apply(lambda x:  
x.skew()).sort_values(ascending=False)  
  
    high_skew = skewed_features[abs(skewed_features) > 0.5]  
    skewed_features = high_skew.index  
  
    # Perform the Box-Cox transformation  
    for column in skewed_features:  
        features[column] = boxcox1p(features[column],  
boxcox_normmax(features[column] + 1))  
  
    return features  
  
features = fix_skew(features)  
features.head()
```

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	LotShape
LandContour \						
0	60	RL	18.144573	13.833055	Pave	Reg
Lvl						
1	20	RL	20.673625	14.117918	Pave	Reg
Lvl						
2	60	RL	18.668046	14.476513	Pave	IR1

```

Lvl
3          70          RL      17.249651  14.106197  Pave      IR1
Lvl
4          60          RL      21.314282  15.022008  Pave      IR1
Lvl

  Utilities LotConfig LandSlope ... SaleCondition HasPool Has2ndFloor
\
0  AllPub      Inside      Gtl ...      Normal      0.0      1
1  AllPub      FR2        Gtl ...      Normal      0.0      0
2  AllPub      Inside      Gtl ...      Normal      0.0      1
3  AllPub      Corner      Gtl ...      Abnorml      0.0      1
4  AllPub      FR2        Gtl ...      Normal      0.0      1

  HasGarage  HasBsmt  HasFireplace  TotalSF  Total_Bathrooms  \
0  0.874318  41.797413              0  35.576148              3.5
1  0.874318  41.797413              1  35.371882              2.5
2  0.874318  41.797413              1  36.241067              3.5
3  0.874318  41.797413              1  35.120719              2.0
4  0.874318  41.797413              1  39.002312              3.5

  Total_porch_sf  YearsSinceRemodel
0      12.339283              5
1       0.000000             31
2      10.090749              6
3      28.165644             36
4      14.606323              8

[5 rows x 84 columns]

features = pd.get_dummies(features).reset_index(drop=True)
features.shape

(2917, 324)

X = features.iloc[:len(y), :]
test = features.iloc[len(y):, :]

X.shape, test.shape, y.shape

((1458, 324), (1459, 324), (1458,))

```

Training and Testing Machine Learning Models

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
0.25, random_state = 0)

from sklearn.metrics import mean_squared_error, mean_absolute_error

def evaluation(model, x_train_std, y_train, x_test, y_test, train =
True):
    if train == True:
        pred = model.predict(x_train_std)

        print("Train Result:\n
=====")
        print(f"Root Mean Squared Error:
{np.sqrt(mean_squared_error(y_train, pred))}")
        print("_____")
        print(f"Mean Squared Error: {mean_squared_error(y_train,
pred)}")
        print("_____")
        print(f"Mean Absolute Error: \n{mean_absolute_error(y_train,
pred)}")
        print("_____")

        print()

    if train == False:
        pred = model.predict(x_test)

        print("Test Result:\n
=====")
        print(f"Root Mean Squared Error:
{np.sqrt(mean_squared_error(y_test, pred))}")
        print("_____")
        print(f"Mean Squared Error: {mean_squared_error(y_test,
pred)}")
        print("_____")
        print(f"Mean Absolute Error: \n{mean_absolute_error(y_test,
pred)}")
        print("_____")
```

Lasso Regression

```
from sklearn.linear_model import Lasso
ls = Lasso(alpha = 0.1)
ls.fit(X_train, y_train)

evaluation(ls, X_train, y_train, X_test, y_test, True)
evaluation(ls, X_train, y_train, X_test, y_test, False)
```

Train Result:

=====

Root Mean Squared Error: 0.17406961837286633

Mean Squared Error: 0.030300232040475328

Mean Absolute Error:
0.12101728094302051

Test Result:

=====

Root Mean Squared Error: 0.15620930593614293

Mean Squared Error: 0.024401347261051497

Mean Absolute Error:
0.11449019184489485

Ridge Regression

```
from sklearn.linear_model import Ridge  
ridge = Ridge(alpha = 0.1)  
ridge.fit(X_train, y_train)
```

```
evaluation(ridge, X_train, y_train, X_test, y_test, True)  
evaluation(ridge, X_train, y_train, X_test, y_test, False)
```

Train Result:

=====

Root Mean Squared Error: 0.08610797525850908

Mean Squared Error: 0.007414583403120012

Mean Absolute Error:
0.05979100494611573

Test Result:

=====

Root Mean Squared Error: 0.11356238712731023

Mean Squared Error: 0.012896415770053078

Mean Absolute Error:
0.08087795398725854

XGB Regressor

```
import xgboost as xgb
xgb_regress = xgb.XGBRegressor(n_estimators = 1000, learning_rate = 0.1)
xgb_regress.fit(X_train, y_train)

evaluation(xgb_regress, X_train, y_train, X_test, y_test, True)
evaluation(xgb_regress, X_train, y_train, X_test, y_test, False)
```

Train Result:

=====

Root Mean Squared Error: 0.0014216860482755428

Mean Squared Error: 2.021191219861329e-06

Mean Absolute Error:
0.0009820525604962483

Test Result:

=====

Root Mean Squared Error: 0.11682680271424793

Mean Squared Error: 0.013648501832433808

Mean Absolute Error:
0.08398457096015714

Submission

```
test_pred = ridge.predict(test)
submission = pd.DataFrame(test_id, columns = ['Id'])
test_pred = np.expml(test_pred)
submission['SalePrice'] = test_pred
submission.head()
```

	Id	SalePrice
0	1461	118633.420153
1	1462	162220.710382
2	1463	189656.747181
3	1464	203191.189164
4	1465	205856.623664

```
submission.to_csv("result_ridge.csv", index = False, header = True)
```

Hope you liked the notebook, any suggestions would be highly appreciated.

I will continue experimenting in future versions of the notebook.

Please upvote if you liked it.